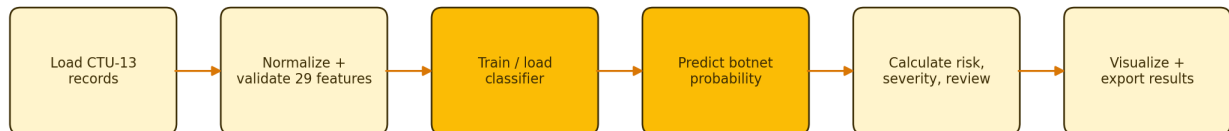


Botnet Traffic Detection

Complete Project Documentation

Operational Detection Workflow



A Windows-friendly defensive machine-learning application that detects botnet traffic from real CTU-13-derived network-flow records. The package includes a Streamlit dashboard, pretrained model, source dataset files, preprocessing utilities, command-line workflows, automated tests, and exportable analyst reports.

Dataset source: CTU-13, Stratosphere IPS / Czech Technical University in Prague

Table of Contents

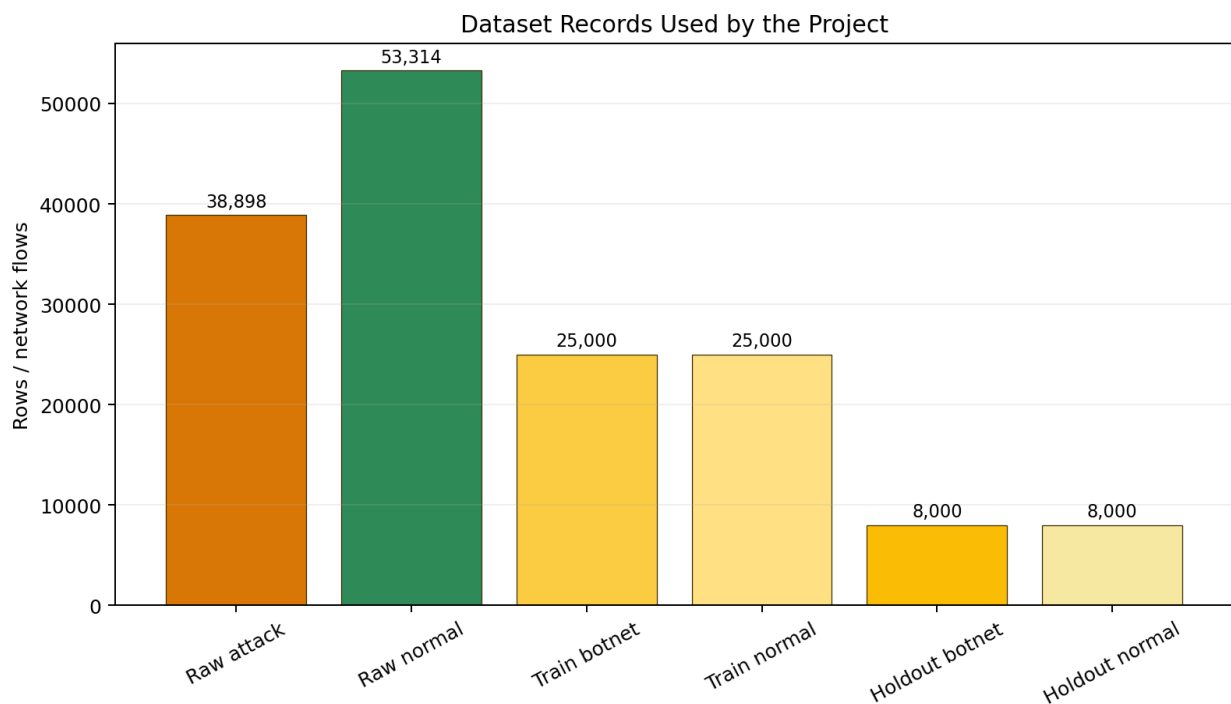
- 1. Executive summary
- 2. Problem statement and objectives
- 3. Dataset and provenance
- 4. Data files and processing strategy
- 5. Feature schema
- 6. System architecture
- 7. Dashboard features
- 8. Machine-learning pipeline
- 9. Model evaluation
- 10. Inference, confidence, and risk logic
- 11. Installation on Windows
- 12. User guide
- 13. Command-line workflows
- 14. Testing and validation
- 15. Responsible use and security considerations
- 16. Limitations and roadmap
- 17. Project structure and reference links

This documentation describes the project itself, its data, implementation, features, operation, and limitations.

1. Executive summary

Botnet Traffic Detection is a portfolio-ready binary network-flow classifier. It predicts whether each compatible record is Normal or Botnet. The application is local-first, uses open-source data instead of generated traffic, and provides an end-to-end workflow from dataset ingestion to model export and incident reporting.

- Binary classification: Normal versus Botnet.
- Real CTU-13-derived CSV data is bundled in the project.
- Pretrained Extra Trees model for immediate demonstration.
- Alternative training options: Random Forest and Logistic Regression.
- Streamlit dashboard with light and dark yellow-white themes.
- Batch detection, manual flow assessment, metrics, feature importance, and exports.



2. Problem statement and objectives

Botnet activity can create automated communication patterns, scanning behavior, command-and-control traffic, and repeated connections. Manual review of large flow datasets is slow. This project demonstrates how machine learning can prioritize suspicious flow records for analyst review.

Objectives

- Build a reproducible defensive classification workflow on real open data.
- Expose the complete machine-learning lifecycle in a visual dashboard.
- Provide transparent evaluation instead of displaying predictions only.
- Convert probabilities into analyst-oriented risk and review indicators.
- Package the project so it can run locally on Windows.

Non-goals

- Replacing an enterprise IDS, SIEM, or packet-inspection platform.
- Performing active scanning or exploitation.
- Claiming that a high model score proves compromise.
- Guaranteeing generalization to every modern network environment.

3. Dataset and provenance

The project uses CTU-13, a labeled botnet-traffic dataset published by Stratosphere IPS / Czech Technical University in Prague. The official collection contains thirteen scenarios with real botnet traffic mixed with normal and background traffic.

Item	Description
Official dataset	CTU-13 botnet traffic dataset
Publisher	Stratosphere IPS / Czech Technical University in Prague
Official page	https://www.stratosphereips.org/datasets-ctu13
CSV conversion	https://github.com/imfaisalmalik/CTU13-CSV-Dataset
Conversion method	CTU-13 PCAP records processed with CICFlowMeter
Project labels	0 = Normal, 1 = Botnet

The dataset is not covered by the project MIT software license. Users must review the original dataset terms, citation guidance, and redistribution requirements.

Data authenticity

- The project does not generate artificial network-flow rows.
- Raw attack and normal CSV files come from the referenced open-source conversion repository.
- Processed files are produced by selecting, normalizing, balancing, and splitting existing rows.

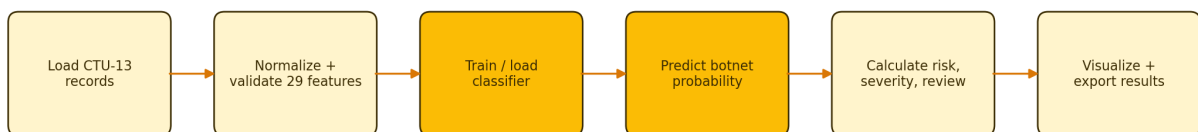
4. Data files and processing strategy

File	Rows	Columns	Purpose
CTU13_Attack_Traffic.csv	38,898	59	Raw botnet/attack flows
CTU13_Normal_Traffic.csv	53,314	59	Raw normal/background flows
ctu13_train_balanced.csv	50,000	30	Balanced training sample: 25,000 per class
ctu13_holdout_labeled.csv	16,000	30	Independent labeled validation sample
ctu13_holdout_unlabeled.csv	16,000	29	Batch detection demonstration input

Processing steps

- Read the attack and normal CSV files.
- Normalize column names into project-compatible snake_case names.
- Select the 29 supported numerical features.
- Replace infinite values with missing values.
- Apply median imputation inside the scikit-learn pipeline.
- Create balanced class samples for responsive local training.
- Separate training and holdout records to avoid immediate reuse of the same rows.

Operational Detection Workflow



5. Feature schema

The model expects 29 normalized numeric features. User-uploaded CSV files must provide all required features, while labeled training files must also contain the `label` column.

Feature	Description
flow_duration	Total duration of the flow.
tot_fwd_pkts	Total packets in the forward direction.
tot_bwd_pkts	Total packets in the backward direction.
totlen_fwd_pkts	Total forward-packet payload length.
totlen_bwd_pkts	Total backward-packet payload length.
fwd_pkt_len_max	Maximum forward-packet length.
fwd_pkt_len_mean	Average forward-packet length.
bwd_pkt_len_max	Maximum backward-packet length.
bwd_pkt_len_mean	Average backward-packet length.
flow_byts_s	Flow throughput in bytes per second.
flow_pkts_s	Flow packet rate per second.
flow_iat_mean	Mean inter-arrival time for the flow.
flow_iat_std	Standard deviation of flow inter-arrival times.
flow_iat_max	Maximum flow inter-arrival time.
fwd_iat_mean	Mean forward-direction inter-arrival time.

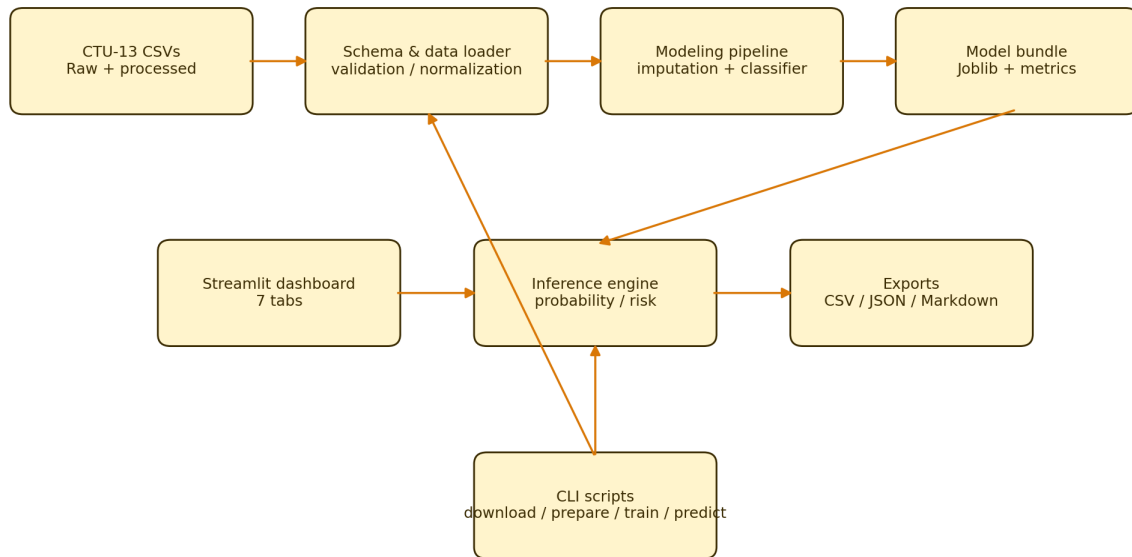
5. Feature schema - continued

Feature	Description
bwd_iat_mean	Mean backward-direction inter-arrival time.
fwd_pkts_s	Forward packet rate per second.
bwd_pkts_s	Backward packet rate per second.
pkt_len_mean	Mean packet length across the flow.
pkt_len_std	Standard deviation of packet length.
syn_flag_cnt	Number of TCP SYN flags.
rst_flag_cnt	Number of TCP RST flags.
ack_flag_cnt	Number of TCP ACK flags.
down_up_ratio	Download-to-upload traffic ratio.
pkt_size_avg	Average packet size.
init_bwd_win_byts	Initial backward TCP-window size in bytes.
fwd_act_data_pkts	Forward packets carrying actual data.
active_mean	Mean active-period duration.
idle_mean	Mean idle-period duration.

Feature values are flow statistics, not packet payload content. This keeps the project focused on flow-level behavior and reduces exposure to sensitive application data.

6. System architecture

CTU-13 Botnet Traffic Detection - Software Architecture



Component	Responsibility
app.py	Streamlit interface, tabs, session state, charts, and downloads.
src/data_loader.py	Load, normalize, label, combine, and balance CTU-13 records.
src/schema.py	Define required features and validate uploaded datasets.
src/modeling.py	Build classifiers, train models, calculate metrics, and save bundles.
src/inference.py	Generate predictions, probabilities, confidence, risk, severity, and review status.
src/exporter.py	Create CSV, JSON, and Markdown export artifacts.
scripts/	Command-line entry points for download, preparation, training, and prediction.

7. Dashboard features

Tab	Function
Overview	Shows training rows, classes, active model, dataset, and project capabilities.
Dataset	Loads the balanced sample, full bundled CSV files, or a compatible uploaded CSV.
Train & evaluate	Trains a chosen classifier and displays metrics, confusion matrix, report, and feature importance.
Batch detection	Scores many unlabeled flow records and supports report export.
Manual flow check	Allows ad hoc entry of feature values for one flow.
Model insights	Explains selected features and interpretation constraints.
Dataset provenance	Displays dataset sources, citation guidance, and data-origin notes.

Interface themes

- Light theme: near-white background, yellow accent, dark-brown text.
- Dark theme: deep brown-black background, gold-yellow accent, warm white text.
- The launcher ``run_app.bat`` lets the user choose either theme.

8. Machine-learning pipeline

Model	Characteristics
Extra Trees	Bundled default; many randomized trees, nonlinear interactions, fast inference, native feature
Random Forest	Bagged decision trees with class balancing; robust general-purpose baseline.
Logistic Regression	Linear probabilistic baseline; includes standard scaling and is easier to interpret.

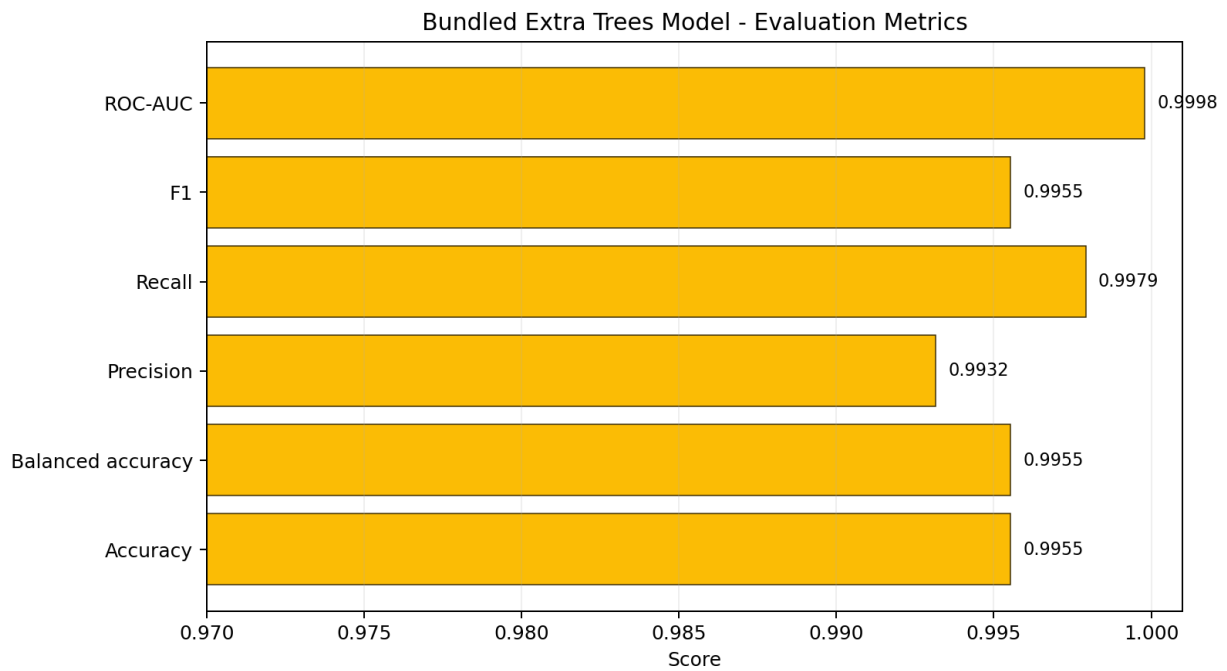
Training sequence

- Validate all required features and binary labels.
- Build the feature matrix and target vector.
- Create a stratified train/test split.
- Apply median imputation for missing values.
- Apply StandardScaler only for Logistic Regression.
- Fit the selected classifier.
- Calculate classification metrics and probability-based ROC-AUC.
- Calculate permutation feature importance.
- Optionally perform stratified 3-fold cross-validation.
- Save the fitted pipeline and metadata as a Joblib bundle.

9. Model evaluation

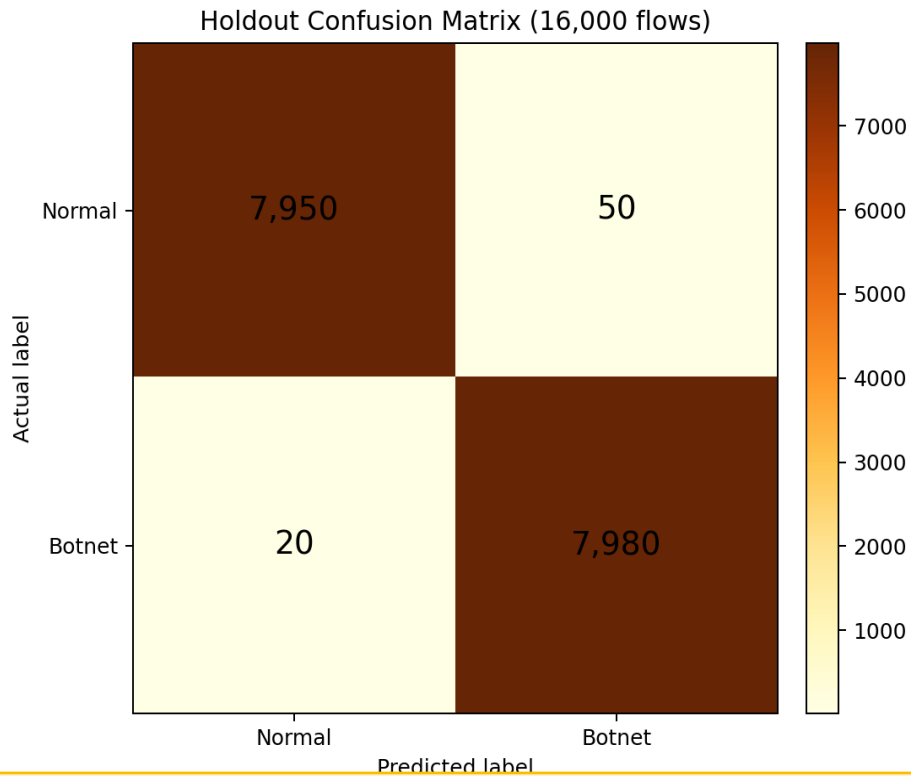
The bundled model is an Extra Trees classifier trained with 50,000 balanced CTU-13-derived records. The package stores the following evaluation metrics:

Metric	Score	Interpretation
Accuracy	0.9955	Overall correct predictions.
Balanced accuracy	0.9955	Average recall across both classes.
Precision	0.9932	Share of predicted botnets that are botnets.
Recall	0.9979	Share of botnet records detected.
F1-score	0.9955	Harmonic balance of precision and recall.
ROC-AUC	0.9998	Ranking quality across thresholds.



9. Model evaluation - holdout analysis

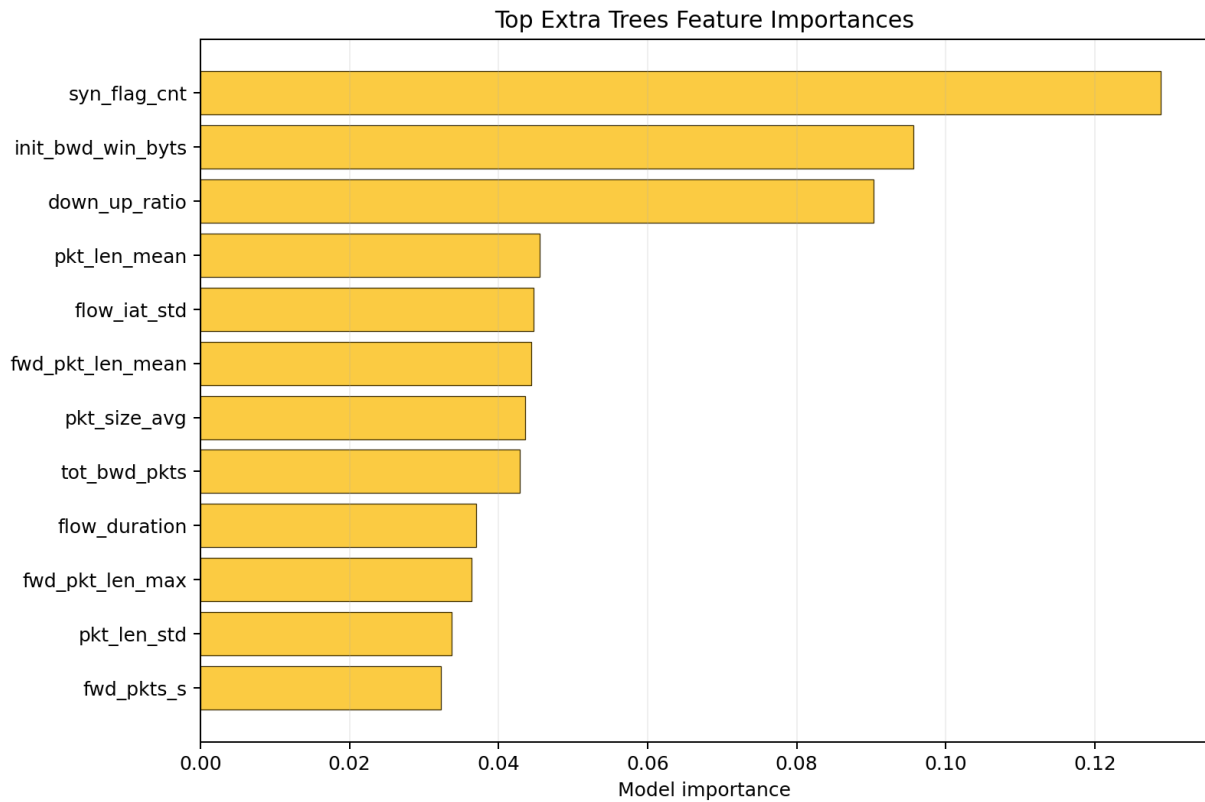
The included labeled holdout contains 16,000 records: 8,000 Normal and 8,000 Botnet. The bundled model produced 7,950 true negatives, 50 false positives, 20 false negatives, and 7,980 true positives on this holdout.



High scores inside CTU-13-derived splits do not guarantee equal performance on modern or unrelated enterprise networks. Dataset age, flow exporter behavior, traffic mix, and collection conditions may cause performance changes.

9. Model evaluation - feature importance

The chart below shows the internal feature importance of the bundled Extra Trees classifier. Importance represents model dependence, not causal proof that a feature indicates botnet behavior.



10. Inference, confidence, and risk logic

For each compatible flow, the pipeline calculates a botnet probability. A probability of at least 0.5 is mapped to Botnet; otherwise, the output is Normal. Confidence is the larger probability between the predicted class and the alternative class.

Output	Rule
prediction	Botnet when probability ≥ 0.5 ; otherwise Normal.
confidence	$\max(\text{botnet_probability}, 1 - \text{botnet_probability})$
risk_score	$\text{botnet_probability} \times 100$
severity	Low < 40 ; Medium 40-64.99; High 65-84.99; Critical ≥ 85
review_status	Needs Review when confidence is below the configured threshold.

Analyst interpretation

- Risk score is a transformed model probability, not an independent threat-intelligence score.
- Low-confidence predictions are explicitly marked for review.
- Operational action should require correlation with logs, asset context, and other security controls.

11. Installation on Windows

System requirements

- Windows 10 or Windows 11.
- Python 3.11 or Python 3.12.
- Approximately several gigabytes of free space for environment and data.
- Internet connection is needed only when installing dependencies or re-downloading data.

Automatic installation

- Install Python and enable Add Python to PATH.
- Extract the ZIP package.
- Double-click `setup\_windows.bat`.
- After installation completes, run `run\_app.bat`.
- Choose 1 for Dark theme or 2 for Light theme.

Manual PowerShell installation

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
streamlit run app.py
```

If PowerShell blocks activation, use `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` for the current window.

12. User guide

Immediate batch detection

- Run the application and open Batch detection.
- Keep Bundled CTU-13 holdout selected.
- Click Analyze network flows.
- Review prediction counts, probability, confidence, risk score, severity, and review status.
- Download the CSV predictions or Markdown incident report.

Training a new model

- Open Dataset and select a bundled or uploaded labeled dataset.
- Click Use this training dataset.
- Open Train & evaluate.
- Choose Extra Trees, Random Forest, or Logistic Regression.
- Configure test split, random seed, and optional 3-fold cross-validation.
- Click Train and evaluate model.
- Inspect metrics, confusion matrix, classification report, and feature importance.
- Download the model or metrics JSON.

Manual assessment

Manual flow check provides input controls for the supported features. It is useful for demonstrating model behavior but is not intended as a substitute for automated data collection.

13. Command-line workflows

Re-download and prepare dataset

```
python scripts\download_dataset.py  
python scripts\prepare_data.py
```

Train the default model

```
python scripts\train_model.py --model "Extra Trees"
```

Predict an unlabeled CSV file

```
python scripts\predict_csv.py --model models\ctu13_botnet_detector.joblib --input  
data\processed\ctu13_holdout_unlabeled.csv --output reports\ctu13_predictions.csv
```

Windows helper files

File	Purpose
download_dataset.bat	Download the referenced CSV dataset and run preparation.
prepare_data.bat	Recreate balanced and holdout files.
train_model.bat	Train and save the default model.
run_tests.bat	Run the automated unit tests.
run_app.bat	Open the theme selector and launch Streamlit.

14. Testing and validation

Test	Purpose
test_schema	Verifies required features and binary labels.
test_training_and_inference	Trains a small Extra Trees model and validates output ranges.
test_model_persistence	Saves and reloads the Joblib model bundle.
test_severity	Checks Low, Medium, High, and Critical thresholds.

Run tests

```
python -m unittest discover -s tests -v
```

- The package includes four automated tests.
- Tests use a small balanced subset to remain fast on ordinary laptops.
- Passing tests verify software behavior, not operational cybersecurity effectiveness.

15. Responsible use and security considerations

- Analyze only network data that you are authorized to access.
- Do not treat prediction output as proof of malicious activity.
- Correlate results with firewall, endpoint, DNS, authentication, and asset-context logs.
- Protect uploaded flow files because metadata can still reveal sensitive infrastructure patterns.
- Do not expose the Streamlit development server directly to the public internet.
- Review the original CTU-13 terms before redistribution, publication, or commercial use.
- Document model version, threshold, dataset, and analyst decision for auditability.

The project is defensive and does not include attack execution, exploitation, credential collection, or packet-payload inspection features.

16. Limitations and roadmap

Current limitations

- CTU-13 is an older dataset and may not represent current botnet behavior.
- CSV conversion and flow-exporter settings can influence feature distributions.
- Balanced training data does not reflect every operational class ratio.
- The binary label merges diverse attack patterns into a single Botnet class.
- There is no live packet or NetFlow collector in the MVP.
- There is no authentication, persistent database, model registry, or production monitoring.
- Model probability may be poorly calibrated outside the training distribution.

Roadmap

- Scenario-aware splitting to reduce leakage between related captures.
- Cross-dataset evaluation using more recent intrusion datasets.
- Probability calibration and precision-recall threshold analysis.
- SHAP or local explanation views for individual predictions.
- Data-drift and model-performance monitoring.
- Persistent experiment tracking and model registry.
- REST API deployment and controlled SIEM sandbox integration.
- Support for streaming flow records through a message queue.

17. Project structure and reference links

```
botnet-traffic-detection/  
|-- app.py  
|-- src/  
|-- scripts/  
|-- data/raw/  
|-- data/processed/  
|-- models/ctu13_botnet_detector.joblib  
|-- reports/  
|-- tests/  
|-- docs/  
|-- setup_windows.bat  
|-- run_app.bat  
|-- run_tests.bat  
|-- requirements.txt  
`-- README.md
```

Reference links

Resource	URL
CTU-13 official dataset	https://www.stratosphereips.org/datasets-ctu13
CTU-13 CSV conversion repository	https://github.com/imfaisalmalik/CTU13-CSV-Dataset

Recommended portfolio presentation: show the real-data provenance, run batch detection, explain the confusion matrix and false-positive/false-negative trade-off, then discuss why cross-dataset validation is necessary before operational use.